Tech Spec: Wikipedia Election Scraper (Apify Actor)

Version: 0.2
Creator: Nel Barthelemy
Status: Draft — v1 build scope is `race_candidates` only; `state_all` and `candidate_search` are designed below for v2, not being built yet.
Last Updated: 2026-08-19

1. Overview

A standalone Apify Actor with three structured query modes, all sourced live from Wikipedia. **Only mode 2 is being built for v1** — modes 1 and 3 are designed in full here (they were explored and verified live this session) so v2 is a known quantity, not a redesign, but they are explicitly deferred:

1. `state_all` V2 · DEFERRED — given a state (+ cycle), return every election happening in that state.
2. `race_candidates` V1 · BUILD NOW — given a state + office + cycle, return the current candidate list (name, party, status, short bio) for that specific race. This was the original v0.1 scope.
3. `candidate_search` V2 · DEFERRED — given a candidate name, return which race they're running in plus basic bio info.

Sold on the Apify Store as a paid scraper, targeting developers, researchers, and other applications that need normalized election data without building their own Wikipedia-parsing pipeline.

Query modes are explicit, not free-text. The caller passes a `mode` field plus the params that mode needs (see §5). No natural-language prompt parsing — that was considered and rejected: it would add a whole parsing/disambiguation layer for no real benefit, since API consumers can trivially pass structured fields.

Relationship to the Political Transparency App: this reuses the Wikipedia heading-parse pipeline designed and live-tested for that project's Phase 1 nominee-resolution feature (see `tech-spec.md` §3, §6), but is a **separate, standalone product** — no shared database, no FEC cross-referencing, no shared deployment. It can ship before that app's Phase 1 is finished.

Key divergence from the Political Transparency App: that app permanently excludes party affiliation from all UI and payloads as a core product principle. This scraper does **not** carry that constraint — its customers are developers who may have legitimate reasons to want party data, and Wikipedia's own structure (party-labeled headings) makes it nearly free to include. `party` is a first-class output field here. This is a deliberate, explicit divergence, not an oversight — worth remembering if code or docs are ever shared between the two projects.

2. Goals

v1 (build now):

- Given a state, office, and election cycle, return the current candidate list for that race (`race_candidates`)
- Source data live from Wikipedia (MediaWiki Action API), not a static/stale copy
- Distinguish real, current candidates (`declared` / `runoff_pending` / `nominee`) from primary losers, withdrawals, and other dropped statuses — same filtering logic as the main app
- Include `party` per candidate (see divergence note above), with true independents represented as `party: null`, not a fabricated value
- Cheap enough per-query (via caching) to be viable at Apify's pay-per-event pricing

v2 (designed, deferred):

- Given a state (+ cycle), return every election taking place in that state (`state_all`)
- Given a candidate name, return which race they're running in plus basic bio info (`candidate_search`)

3. Scope

In scope for v1:

- Single query mode: `race_candidates` (§5, §6.2, §7.2). Explicit `mode` field is still part of the input shape even with one mode, so v2 can add modes without a breaking change (§5).
- **U.S. Senate only.** Expanding to House/Governor/state legislature/local requires new title-template patterns and testing per office type that don't exist yet — revisit once Senate coverage is proven across more than the ~4 races spot-checked so far (§8).
- Current election cycle only (2026) — title pattern is unverified for past cycles.
- Regular and special elections (confirmed live for the 2026 FL/OH specials)
- Runoff handling (`runoff_pending` status, `runoff_date` field)

Designed but explicitly deferred to v2 (not being built now — see §6.1, §6.3, §7.1, §7.3 for the full designs, kept here so v2 doesn't require re-deriving them):

- `state_all` — all-offices-in-a-state lookup.
- `candidate_search` — candidate-name lookup.

Explicitly out of scope, no version targeted:

- Non-Senate offices for `race_candidates` and `candidate_search` (House, Governor, state legislature, local) — planned expansion, not designed yet.
- FEC cross-referencing / candidate ID matching — this product is Wikipedia-only by design.
- Historical (pre-2026) cycles.

4. Data Source

MediaWiki Action API (`en.wikipedia.org/w/api.php`) — free, public, no API key. Same two endpoints already live-verified against real 2026 Senate race data this session:

- `action=query&prop=extracts&explaintext=1&titles={title}` — primary lookup. Article text is heading-structured (`== {Party} primary ==` → `=== Candidates ===` → `==== Nominee/Declared/Advanced to runoff/... ====`) and machine-parseable.
- `action=query&list=search&srsearch={query}` — fallback when the constructed title doesn't resolve directly (confirmed necessary for some title variants); also the primary lookup path for `candidate_search` (§6.3), where the title isn't known in advance.
- `action=query&prop=categories&cllimit=500` — used by `candidate_search` to confirm a found page is actually a `{cycle} United States Senate` candidate (via `Category:Candidates in the {cycle} United States Senate elections`) before trusting the page as a match. **Confirmed live**: this category exists and is *cycle-wide*, not per-state — it cannot tell you *which* state's race a candidate is in, only that they're in one somewhere. State must come from intro-text extraction instead (§6.3).
- `action=parse&prop=wikitext&section=0` — used by `candidate_search` to pull the infobox for `party` (`political_party` field). **Confirmed live** the infobox does *not* expose the race/state/office — only the article's prose does (see §6.3).

Licensing: CC BY-SA 4.0. Commercial use is permitted. Extracted *facts* (candidate name, status, party) can be sold freely; if any raw article text is ever returned verbatim (e.g. a bio sentence), the output must carry attribution (`source_url` , `source_license`) — already built into the output schema below, not an afterthought.

Etiquette: requests must set a descriptive `User-Agent` identifying this product, per Wikipedia's API usage expectations — same practice used throughout this session's testing.

5. Input Schema

Discriminated on `mode` . Only `race_candidates` is implemented in v1; `state_all` and `candidate_search` are shown for the v2 design and should not be built/exposed yet.

```
{ "mode": "race_candidates", "state": "Michigan", "office": "senate", "cycle": 2026 }
```

```
// v2 — designed, not built
{ "mode": "state_all", "state": "Michigan", "cycle": 2026 }
```

```
// v2 — designed, not built
{ "mode": "candidate_search", "name": "Abdul El-Sayed", "cycle": 2026 }
```

`state` accepts full names or 2-letter codes, normalized via a state-code ↔ full-name crosswalk (same lookup already flagged as needed infrastructure in the main app's spec, §12). `cycle` on `candidate_search`

scopes which "Candidates in the {cycle} ..." category to check (§6.3) — without it there's no way to confirm the match is current vs. a past cycle.

6. Pipelines

6.2 `race_candidates` V1 · BUILD TARGET

(This is the original v0.1 pipeline — Senate-only, unchanged. Renumbered to 6.2 to stay aligned with §5/§7 across both v1 and v2 modes.)

1. Normalize `state` to its full name.
2. Check a maintained per-cycle config (e.g. `{ 2026: ["FL", "OH"] }`) to determine whether this state's race is a special election, which changes the title template.
3. Construct the Wikipedia title deterministically:
   - Regular: `"{cycle} United States Senate election in {state}"`
   - Special: `"{cycle} United States Senate special election in {state}"`
4. Fetch via `prop=extracts`. If the title 404s, fall back to `list=search` and flag the result as `title_matched_via: "search_fallback"` so a low-confidence match is distinguishable from a direct hit.
5. Heading-parse the article text: split on `==` / `===` / `====` boundaries, walk into each party's `=== Candidates ===` subsection, and classify each listed candidate by their status heading.
6. Keep only candidates under `Declared`, `Advanced to runoff`, or `Nominee` headings. Drop everyone else (`Declined`, `Withdrawn`, `Did not file`, `Not on ballot`, `Disqualified`, `Eliminated in primary`, `Ineligible`) — same rule as the main app.
7. Record each kept candidate's party from the heading it was found under. For the `Independents` subsection specifically, set `party: null` rather than inventing one.
8. Extract `primary_date` (and `runoff_date`, if applicable) from the intro/background text. Two known sentence patterns exist — a single clean sentence for non-runoff states, a longer compound sentence for runoff states (confirmed live for Oklahoma) — extraction must handle both, not just the simpler one.
9. Cache the parsed result, keyed on `(state, office, cycle)`, with a TTL that expires once that race's own recorded primary/runoff date has passed. This keeps Wikipedia request volume reasonable and keeps marginal cost per paid Apify run close to zero.
10. Return the structured result (§7.2).

6.1 `state_all` V2 · DEFERRED

1. Normalize `state` to its full name.
2. Construct the Wikipedia title: `"{cycle} {state} elections"` (confirmed live for Michigan — pageid 74607542, clean `==`-level headings: Federal → US Senate / US House, Executive → Governor /

Secretary of State / Attorney General, Legislature → State Senate / State House, Judiciary → Supreme Court, plus local boards and ballot initiatives).

3. Fetch via `prop=extracts`. If the title 404s, fall back to `list=search`, same `title_matched_via` flagging as §6.2 step 4. **Not yet verified whether this title pattern holds for every state** — only Michigan spot-checked (§8).
4. Heading-parse into a flat list of `{ office, race_title, source_url }` entries — one per race/section found (Senate, House, Governor, etc.), not full candidate lists.
5. For any entry where `office == "senate"`, additionally set `resolvable_via: "race_candidates"` so callers know that race can be queried in full via mode 2. All other offices are out of scope for full resolution per §3 — return title/link only.
6. Cache keyed on `(state, cycle)`.
7. Return the structured result (§7.1).

6.3 `candidate_search` V2 · DEFERRED

1. Attempt a direct title fetch on `name` via `prop=extracts`. If it 404s or disambiguates, fall back to `list=search&srsearch={name}` and take the top hit, flagging `title_matched_via: "search_fallback"`.
2. Fetch `prop=categories` on the resolved page. Confirm membership in `Category:Candidates in the {cycle} United States Senate elections`. **If absent, return a `not_found` / `no_match` result rather than guessing** — this category check is what keeps the mode Senate-only per §3 and prevents returning a stale/irrelevant person page.
3. Extract the race from intro prose via regex, matching the same two title patterns as §6.2 step 3 (regular and special): `"{cycle} United States Senate (special )?election in ({state})"`. **Confirmed live**: this sentence pattern is present and reliable (verified on Abdul El-Sayed's page: "...the Democratic nominee in the 2026 United States Senate election in Michigan"). If no match is found in the intro, scan the full extract before failing.
4. Once `state` is resolved, optionally re-run §6.2 to get the candidate's current `status` within that race (the candidate's own page doesn't reliably state their status — e.g. "nominee" vs. "declared" — as cleanly as the race page does).
5. Pull `party` from the infobox (`political_party` field via `action=parse&prop=wikitext`), normalized the same way as §6.2's heading-derived party values for consistency.
6. Cache keyed on `(name, cycle)`.
7. Return the structured result (§7.3).

7. Output Schema

Each mode returns `source_license` (`"CC BY-SA 4.0"`), `title_matched_via` (`"direct"` | `"search_fallback"`), `parse_confidence` (`"high"` | `"needs_review"`), and `last_updated`, in addition to the mode-specific fields below. `parse_confidence` is `high` for a direct title match with cleanly-

headed/parseable content, `needs_review` when the search fallback was used or the expected structure didn't match cleanly.

7.2 `race_candidates` V1 · BUILD TARGET


```
{
  "query": { "state": "Michigan", "office": "senate", "cycle": 2026 },
  "source_url": "https://en.wikipedia.org/wiki/2026_United_States_Senate_election_in_Michigan",
  "title_matched_via": "direct",
  "primary_date": "2026-08-04",
  "runoff_date": null,
  "candidates": [
    { "name": "Abdul El-Sayed", "party": "Democratic", "status": "nominee", "bio": "Former Wayne
    { "name": "Mike Rogers", "party": "Republican", "status": "nominee", "bio": "Former U.S. Rep
  ]
}
```


`status` values: `declared` | `runoff_pending` | `nominee`.

7.1 `state_all` V2 · DEFERRED


```
{
  "query": { "state": "Michigan", "cycle": 2026 },
  "source_url": "https://en.wikipedia.org/wiki/2026_Michigan_elections",
  "races": [
    { "office": "senate", "race_title": "United States Senate", "resolvable_via": "race_candidat
    { "office": "house", "race_title": "United States House of Representatives", "resolvable_via
    { "office": "governor", "race_title": "Governor and lieutenant governor", "resolvable_via":
  ]
}
```


7.3 `candidate_search` V2 · DEFERRED


```
{
  "query": { "name": "Abdul El-Sayed", "cycle": 2026 },
  "source_url": "https://en.wikipedia.org/wiki/Abdul_El-Sayed",
  "race": { "office": "senate", "state": "Michigan", "election_type": "regular" },
  "party": "Democratic",
  "status": "nominee",
  "bio": "Former Wayne County health director...",
  "not_found": false
}
```

`race` is `null` and `not_found: true` when the category check (§6.3 step 2) fails to confirm the person as a current-cycle Senate candidate.

8. Known Risks & Limitations

Carried over from the main app's research, but with a higher bar here since a parsing failure is now a paying customer's broken result, not an internal admin-reviewed data point:

- **Heading structure and status vocabulary aren't perfectly consistent across articles.** Confirmed live: New Hampshire's Republican section had a "Did not file" subsection missing its `====` markup entirely (appeared as bare text). The parser needs a fallback for malformed/missing headings, not pure heading-boundary parsing.
- **Disambiguation risk**: `"{state} Senate election"` (state legislature) vs `"United States Senate election in {state}"` (federal) are different pages with similar names — confirmed live for Michigan and Florida. Title construction must always use the precise federal phrasing.
- **Runoff-state date sentences use different, less predictable phrasing** than non-runoff states — only spot-checked against one example (Oklahoma) so far. Needs testing against a couple more runoff states (Texas, Louisiana) before this is trusted broadly.
- **Special-election detection is manually maintained**, not automatic — the per-cycle config (§6, step 2) needs a human to update it each cycle, since Wikipedia's data doesn't expose "this is a special election" as a queryable flag on its own.
- **Only spot-checked, not exhaustively tested** — three regular-election articles and one runoff article were verified this session. Before charging money per query, worth testing against a wider sample of the full candidate set (all 35 2026 Senate races) to catch parser edge cases before customers do.
- `state_all` **title pattern only verified for one state.** `"{cycle} {state} elections"` worked cleanly for Michigan, but the alternate title guessed first (`"{cycle} United States elections in {state}"`) does **not** exist (confirmed 404). Needs spot-checking across a few more states before trusting the pattern broadly — naming conventions for this kind of umbrella page aren't guaranteed uniform.
- `state_all` **heading structure will vary in office count/order per state** — Michigan had university boards and a ballot proposal that many states won't have. The parser needs to key off known office-name headings (Senate, House, Governor, etc.) rather than assuming a fixed section count/order.
- **No per-state candidate category exists for** `candidate_search` **.** Confirmed live: Wikipedia only has a cycle-wide `Category:Candidates in the {cycle} United States Senate elections` , not one per state. State must come from regex-matching intro prose (§6.3 step 3), which is a softer signal than a category membership check — a candidate whose intro doesn't follow the standard "nominee in the {cycle} ... election in {state}" phrasing will fail to resolve and needs a `needs_review` / `not_found` path rather than a wrong guess.
- **Name collisions/disambiguation** for `candidate_search` are unhandled — a common name could resolve to the wrong Wikipedia page entirely, and the category check (§6.3 step 2) only catches this if

the wrong page also happens to not be a 2026 Senate candidate. Worth testing against at least one deliberately ambiguous name before trusting this mode broadly.

9. Monetization

Sold via the Apify Store. Two pricing models are available to Actor creators:

- **Pay-per-event** — a fixed price for a defined event (e.g. "one race returned").
- **Pay-per-usage** — no creator-set price; users pay Apify directly for compute/proxy/storage.

Pay-per-event is the better fit here, given the low marginal compute cost once caching (§6, step 9) is in place. **Apify's exact revenue-share cut was not found in their public pricing docs as of this session** — needs direct confirmation from Apify before finalizing a price point.

10. Open Questions

- Exact per-query price point — blocked on confirming Apify's revenue-share cut. Three modes may warrant different per-event pricing (`state_all` is a lighter payload than `race_candidates` / `candidate_search`).
- When to expand `race_candidates` / `candidate_search` beyond Senate to House/Governor/etc. (explicitly deferred per §3, not if-ever).
- Whether historical-cycle support (pre-2026) is worth adding, and whether any of the three title construction patterns even hold for older cycles — untested.
- Whether `state_all` should attempt to resolve non-Senate races too once other offices get their own `race_candidates` -equivalent pipelines, or stay link-only for those permanently.